CODE ANALYSIST REPORT

SEPTEMBER 2026



FUNCTIONAL PROGRAMMING

MODUL 1 - Functional Programming using Python

COMPILED BY:

Mohamad Akbar Noviandi

202410370110388

Informatika G

1. Variable

10 Point

Exercise 1 : The Shape-Shifting Variable

Based on the output of the given experiment, explain how Python behaves when variables (`student_id` and `semester`) are reassigned to a completely different data type. Why do you think this flexibility can be both helpful and dangerous?

Screenshot of the answer (must include student ID / NIM)

```
# Exercise 1

# 202410370110388
# redefine variable

# Before redefine variable
student_id = '202410370110388'
semester = False

print("Before:")
print(student_id, type(student_id))
print(semester, type(semester))

# Redefine variable
student_id = 202410370110388
semester = "Semester 4"

print("\nAfter redefine variable:")
print(student_id, type(student_id))
print(semester, type(semester))
```

Before:

```
Before:
202410370110388 <class 'str'>
False <class 'bool'>
```

After redefine variable:

```
After redefine variable:
202410370110388 <class 'int'>
Semester 4 <class 'str'>
```

Explanation :
Karena bahasa pemrograman python bersifat dinamis dan fleksibel python adalah high-level languange jadi dia lebih fleksibel dibandingkan bahasa low-level tapi kekurangannya atau resikonya ketika project yang menggunakan bahasa python ini sudah compleks dan terlalu panjang beresiko salah membuat variable.

2. Data Type INT & Float

15 Point

Exercise 2 : Investigating Int and Float Memory Size

Modify the values in the `Memory Size  Experiment` using another combination number then run the cell again and observe the differences in the memory sizes they needed. Next, perform an experiment for the float data type.

byte	digit
28	-123456789 s/d 123456789
32	-1234567890 s/d 123456789012345678
36	-1234567890123456780 s/d ?
?*	?

*) Byte values are flexible and do not need to be sequential or in multiples of 4

Does the byte size of a float increase as the value grows larger, similar to the int data type? Furthermore, why do int values have varying memory sizes?

Screenshot of the answer (must include student ID / NIM)
#Experiment a (int)

```python
# Exercise 2

# Memory Size Experiment
# 202410370110388

import sys

data1 = 123456
data2 = 202410370110388

print('Value = ', data1)
print('Type  = ', type(data1))
print('Size  = ', sys.getsizeof(data1), 'bytes\n')

print('Value = ', data2)
print('Type  = ', type(data2))
print('Size  = ', sys.getsizeof(data2), 'bytes')
```

```
Value =  123456
Type  =  <class 'int'>
Size  =  28 bytes

Value =  202410370110388
Type  =  <class 'int'>
Size  =  32 bytes
```

Screenshot of the answer (must include student ID / NIM)
#Experiment b (float)

```python
# Exercise 2

# Memory Size Experiment
# 202410370110388

import sys

data1 = 123456.5
data2 = 202410370110388.8

print('Value = ', data1)
print('Type  = ', type(data1))
print('Size  = ', sys.getsizeof(data1), 'bytes\n')

print('Value = ', data2)
print('Type  = ', type(data2))
print('Size  = ', sys.getsizeof(data2), 'bytes')
```

```
Value =  123456.5
Type  =  <class 'float'>
Size  =  24 bytes

Value =  202410370110388.8
Type  =  <class 'float'>
Size  =  24 bytes
```

Explanation :
Ukuran memori pada tipe data float di Python tidak bertambah walaupun nilai datanya berbeda. Berdasarkan percobaan di atas, kedua data tersebut memiliki ukuran memori yang sama, yaitu 24 bytes. Berbeda dengan tipe data int, ukuran memorinya dapat berubah tergantung dari besar nilai bilangan yang disimpan. Semakin besar nilai int, maka memori yang digunakan juga bisa semakin besar. Hal ini karena Python menggunakan sistem penyimpanan int yang dapat menyesuaikan ukuran memori dengan nilai bilangan yang disimpan.

3. Tuple

10 Point
Exercise 3 : Tuple Syntax Challenge

1. Can you identify why Experiment A fails while the second one runs successfully?
2. What does this tell you about how Python handles tuple creation versus tuple modification?

Screenshot of the answer (must include student ID / NIM)

Eksperimen a :

```
# 202410370110388

#Experiment a

screen_resolution = (1920, 1080)
screen_resolution[0] = 2560
```

```
Traceback (most recent call last):
  File "c:\Users\AkbarNov\Pictures\UMM\Semester 5\Pemrograman Fungsional\python\codelab.py", line 45, in <module>
    screen_resolution[0] = 2560
    ~~~~~~~~~~~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

Eksperimen b:

```
# 202410370110388
#Experiment b

screen_resolution = 1920, 1080
print(screen_resolution)
print(type(screen_resolution))
```

```
(1920, 1080)
<class 'tuple'>
```

Explanation :

1. Experiment A tidak berhasil karena pada percobaan tersebut dilakukan perubahan terhadap elemen tuple. Tuple sendiri bersifat *immutable*, sehingga data yang sudah dibuat tidak dapat diubah lagi. Sedangkan Experiment B berhasil karena hanya mendefinisikan tuple tanpa melakukan perubahan pada elemennya.
2. Dalam Python, tuple dapat ditulis menggunakan tanda kurung maupun

tanpa tanda kurung. Yang penting, setiap data di dalamnya dipisahkan menggunakan tanda koma. Setelah tuple dibuat, elemen yang ada di dalamnya tidak dapat diubah secara langsung.

4. Range

10 Point

Exercise 4 : Countdown Range Analysis

Create a countdown sequence using `range()` (without using if-else statements) where the starting point is determined by the last 2 digits of your Student ID (NIM).
For ODD NIM: Count down from your NIM's last 2 digits down to 1, skipping 2 numbers at a time (decrement by -2). Save and display the result as a List.
For EVEN NIM: Count down from your NIM's last 2 digits down to 0, skipping 3 numbers at a time (decrement by -3). Save and display the result as a Tuple.

Explain the logic behind the code you have created.

Screenshot of the answer (must include student ID / NIM)

```
# 202410370110388
countdown = tuple(range(88, -1, -3))

print(countdown)
```

```
$ C:/Users/AkbarNov/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/AkbarNov/Pictures/UMM/Semester 5/Pemrograman
(88, 85, 82, 79, 76, 73, 70, 67, 64, 61, 58, 55, 52, 49, 46, 43, 40, 37, 34, 31, 28, 25, 22, 19, 16, 13, 10, 7, 4, 1)
```

Explanation :

Pada kode tersebut saya menggunakan range(88, -1, -3) untuk menghasilkan angka yang dimulai dari 24 dan berkurang 3 secara bertahap sampai mencapai 0. Setelah itu, hasil dari range dikonversi menjadi tuple menggunakan tuple(), sehingga angka-angka yang dihasilkan tersimpan dalam bentuk tuple. Terakhir, fungsi print() digunakan untuk menampilkan hasilnya.

5. Dictionary

10 Point

Exercise 5 : Working with Dictionaries

1. When you updated the "semester" key in step 3, did Python create a duplicate key or did it overwrite the old value?
2. What is the explicit error message provided by Python when you uncomment and run step 4? What does that error mean?

Screenshot of the answer (must include student ID / NIM)

```python
# Exercise 5
# 202410370110388

# 1. Create a dictionary with your own data
student_info = {
    "name": "Mohamad Akbar Noviandi",
    "nim": "202410370110388",
    "semester": 4
}

# 2. Add a new key-value pair for your "major"
student_info["major"] = "Informatics"

# 3. Try to update an EXISTING key (change semester to 5)
student_info["semester"] = 5

print("Current Dictionary:", student_info)

# 4.  Uncomment the line below to see what happens
print(student_info["gpa"])
```

```
Current Dictionary: {'name': 'Mohamad Akbar Noviandi', 'nim': '202410370110388', 'semester': 5, 'major': 'Informatics'}
Traceback (most recent call last):
  File "c:\Users\AkbarNov\Pictures\UMM\Semester 5\Pemrograman Fungsional\python\codelab.py", line 83, in <module>
    print(student_info["gpa"])
          ~~~~~~~~~~~~^^^^^^^
KeyError: 'gpa'
```

Explanation:

1. Pada Python, jika kita memasukkan nilai baru pada key yang sebelumnya sudah ada, nilai lama akan langsung digantikan dengan nilai yang baru. Python tidak membuat key tambahan. Pada percobaan tersebut, nilai yang awalnya 4 kemudian berubah menjadi 5.
2. Error saat menjalankan print(student_info[“pa”]) terjadi karena key dengan nama pa tidak terdapat di dalam dictionary student_info. Karena key tersebut belum didefinisikan, Python tidak dapat menemukan data yang ingin ditampilkan.

6. Casting

15 Point

Exercise 6 : Data Type Casting

Based on the code execution, why does the NIM string successfully convert while the string `"hello"` fails? Explain the underlying mechanism of Python's `int()` and `float()` functions regarding literals.

Screenshot of the answer (must include student ID / NIM)

```
# Exercise 6
# NIM: 202410370110388

NIM = "202410370110388"


nim_int = int(NIM)
nim_float = float(NIM)

print("NIM (int):", nim_int)
print("Type:", type(nim_int))

print("NIM (float):", nim_float)
print("Type:", type(nim_float))


try:
    print(int("hello"))
except ValueError as e:
    print("Error int:", e)


try:
    print(float("hello"))
except ValueError as e:
    print("Error float:", e)
```

```
NIM (int): 202410370110388
Type: <class 'int'>
NIM (float): 202410370110388.0
Type: <class 'float'>
Error int: invalid literal for int() with base 10: 'hello'
Error float: could not convert string to float: 'hello'
```

Explanation:
String NIM dapat diubah menjadi int maupun float karena isi string tersebut berupa angka yang valid. Walaupun sebelumnya NIM didefinisikan sebagai teks (string), Python tetap bisa mengubahnya ke tipe data angka. Contohnya, int("202410370110388") akan menghasilkan bilangan bulat, sedangkan float("202410370110388") akan menghasilkan bilangan desimal. Sementara itu, string "hello" tidak bisa dikonversi karena berisi huruf dan bukan angka yang dapat diproses oleh int() maupun float(), sehingga akan menghasilkan error.

7. Function

15 Point

Exercise 7 : Fixing Indentation & Function Errors

1. What explicit error message did Python show before you fixed the indentation of the total and return lines?
2. Why is strict indentation so important when defining a function's body in Python compared to other programming languages?

Screenshot of the answer (must include student ID / NIM)
Before:

```
# Exercise 7
# # FIX THIS CODE!
# 202410370110388
calculate_discount(price, discount):
total = price - (price * discount)
return total

    result = calculate_discount(100000, 0.1)
print("Final Price:", result)
```

```
  File "c:\Users\AkbarNov\Pictures\UMM\Semester 5\Pemrograman Fungsional\python\codelab.py", line 113
    calculate_discount(price, discount):
                                        ^
SyntaxError: invalid syntax
```

After:

```python
# 202410370110388

# Write your fixed code here
def calculate_discount(price, discount):
    total = price - (price * discount)
    return total

result = calculate_discount(10000, 0.1)
print("Final Price:", result)
```

```
Final Price: 9000.0
```

Explanation:

1. Error yang muncul pada kode tersebut adalah syntax error. Hal ini terjadi karena penulisan fungsi def, tanda titik dua, dan indentasinya masih belum tepat. Untuk memperbaikinya, penulisan def dan tanda titik dua perlu diperbaiki, kemudian indentasi pada variabel result juga harus disesuaikan agar masuk ke dalam fungsi.
2. Indentasi pada Python sangat penting karena digunakan untuk menunjukkan bagian kode yang termasuk dalam suatu fungsi. Berbeda dengan Java yang menggunakan kurung kurawal untuk menandai blok kode, Python menggunakan indentasi untuk menentukan batas atau bagian dari suatu blok kode.

8. Recursive Function

15 Point

#Exercise 8 : Analyzing a Recursive Function

Explain step-by-step how `total_sum(3)` executes, and describe how the base case stops this function from calling itself forever!

Screenshot of the answer (must include student ID / NIM)

```python
# Exercise 8

#NIM = 202410370110388
def total_sum(n):
    print(f"Current n: {n}")
    if n == 1:
        return 1
    else:
        return n + total_sum(n - 1)

# Running the function
result = total_sum(3)
print("Final Result:", result)
```

```
Current n: 3
Current n: 2
Current n: 1
Final Result: 6
```

Explanation:

Kode tersebut menggunakan fungsi rekursi, yaitu fungsi yang memanggil dirinya sendiri sampai kondisi tertentu terpenuhi. Pada percobaan ini terdapat beberapa output yang dihasilkan:

1. Pada output pertama, saat menjalankan total_sum(3), nilai current yang ditampilkan adalah 3. Karena nilai n belum mencapai 1, fungsi kembali memanggil dirinya dengan nilai n yang dikurangi 1, sehingga proses berikutnya menjadi total_sum(2).
2. Pada output kedua, pemanggilan sebelumnya adalah total_sum(2), sehingga nilai current yang ditampilkan adalah 2. Kondisi n masih belum sama dengan 1, maka fungsi kembali dipanggil dengan nilai n dikurangi 1 dan dilanjutkan ke total_sum(1).
3. Pada output ketiga, fungsi sudah masuk ke total_sum(1) sehingga current yang ditampilkan adalah 1. Kondisi n == 1 terpenuhi, sehingga fungsi berhenti memanggil dirinya sendiri. Kondisi ini disebut sebagai base case dalam fungsi rekursi.
4. Setelah mencapai base case, fungsi mengembalikan nilai 1. Kemudian hasilnya dihitung dari pemanggilan sebelumnya, yaitu total_sum(2) menghasilkan 2 + 1 = 3 dan total_sum(3) menghasilkan 3 + 3 = 6. Jadi, hasil akhirnya adalah 6. Proses tersebut terjadi karena fungsi harus menyelesaikan pemanggilan berikutnya terlebih dahulu sebelum hasilnya dijumlahkan dengan nilai n.

CODE ANALYSIS ASSESSMENT

Code Analysis	Points (Total 10%)
Exercise Implementation	**5**
All exercises are completed correctly (cumulative points from all existing exercises)	100
Exercise is completed, but the code/output is incorrect	-5 (each exercise)
Answer does not include identity (Name/NIM)	-2 (each exercise)
Completed beyond the practicum session hours	0 (each exercise)
Practicum Report	**5**
Uses the official template, all explanation sections are filled in with correct and clear answers.	100
The answer is incorrect or insufficiently accurate.	-5 (each exercise)
The explanation is found to be identical to another practicant's	0 (each exercise)
Report submitted after the practicum session (overtime)	-40 (from the final score)
Not submitted	0